Low-Data Face Recognition Recognition with YOLOv5

Mateo Monsalve Valencia
Ingeniería de Sistemas e Informática
Ingeniería en Ciencias de Datos
Universidad Pontificia Bolivariana
Medellin, Colombia
mateo.monsalvev@upb.edu.co

I. Introducción

Currently, most neural network based facial recognition systems are built around CNN or embedding based models. Those architectures work very well when the face is already cropped and aligned, but the problem becomes harder when the image also contains background, unknown people, different scales, or partial occlusions [1]–[3]. In this project the goal was not only to recognize a face, but to do it directly from full camera frames, so the model had to learn localization and identity prediction at the same time.

For that reason, this work explored the YOLO family as an end-to-end alternative. YOLO was originally designed for real-time object detection, where the model predicts bounding boxes and classes in a single pass through the image [4]. The final model that was actually trained and deployed in this repository is a `yolov5n` configuration fine-tuned to recognize ten student identities from 640×640 images. The selected checkpoint comes from the final run that included background negatives and stranger-face negatives, and it was later used both as a PyTorch checkpoint (`best.pt`) for the local application and as an exported ONNX model (`best.onnx`) for deployment. This choice was motivated by the need for a lightweight detector with fast inference and enough capacity to preserve general visual features while adapting the classification head to a very small dataset [5]–[7].

Because the dataset is extremely small, the main challenge of the project was not only localization but also overfitting and false positives. The final training strategy therefore focused on making a small detector behave as a few-shot identity recognizer. The first ten layers of the backbone were frozen, the classification loss was increased, context-breaking augmentations such as Mosaic and MixUp were removed, and explicit negative samples were added so the network could learn when no registered identity should be predicted. In other words, the model was trained not only to answer who appears in the image, but also when the correct answer is nobody from the enrolled set. The following sections describe the dataset built for that purpose and the augmentation pipeline used to make the final model viable.

II. Dataset

The base dataset was built from reviewed images of ten identities. Each positive image was resized and padded to `640×640`, converted to RGB JPEG, and assigned one YOLO bounding box around the target person. After the review process, one exact duplicate was excluded and the labeled subset contained 44 positive images in total. The split used for training preserved 34 positive images for the train set and 10 positive images for the validation set, effectively leaving one validation example per identity.

The first experiments showed that the model tended to assign one of the known names to almost any face-like pattern. To reduce that behavior, the training split was expanded with explicit negative samples. Two types of negatives were included: fake faces, which are portraits of people outside the 10 enrolled classes, and empty scenes, which contain no target person at all. These images were stored with empty YOLO label files, so they act as rejection examples rather than as a new class. In the final base dataset, the train split contained 15 negative samples: 10 fake faces and 5 empty scenes. Fig. 1 shows representative positive and negative samples from that curated base dataset.

III. Data Augmentation Pipeline

The augmentation strategy was divided into an offline stage and a lighter online stage. The offline stage was implemented with Albumentations [8] and applied only to positive training images that already contained a valid bounding box. The script combines affine transformations, horizontal flips, hue and saturation shifts, brightness and contrast changes, grayscale conversion, dropout based occlusions, and optional blur, noise, or perspective effects. Bounding boxes were filtered with a minimum visibility threshold of 0.35, so each saved sample remained a valid YOLO training example. With 30 valid variants per positive image, the 34 original positive training samples produced 1020 additional positive images.

Fig. 2 shows the construction of the full data pipeline starting from the 44 reviewed positive images. Those images are first split into 34 labeled training positives and 10 labeled validation positives. Only the 34 training positives are retained and augmented to create the 1069-image

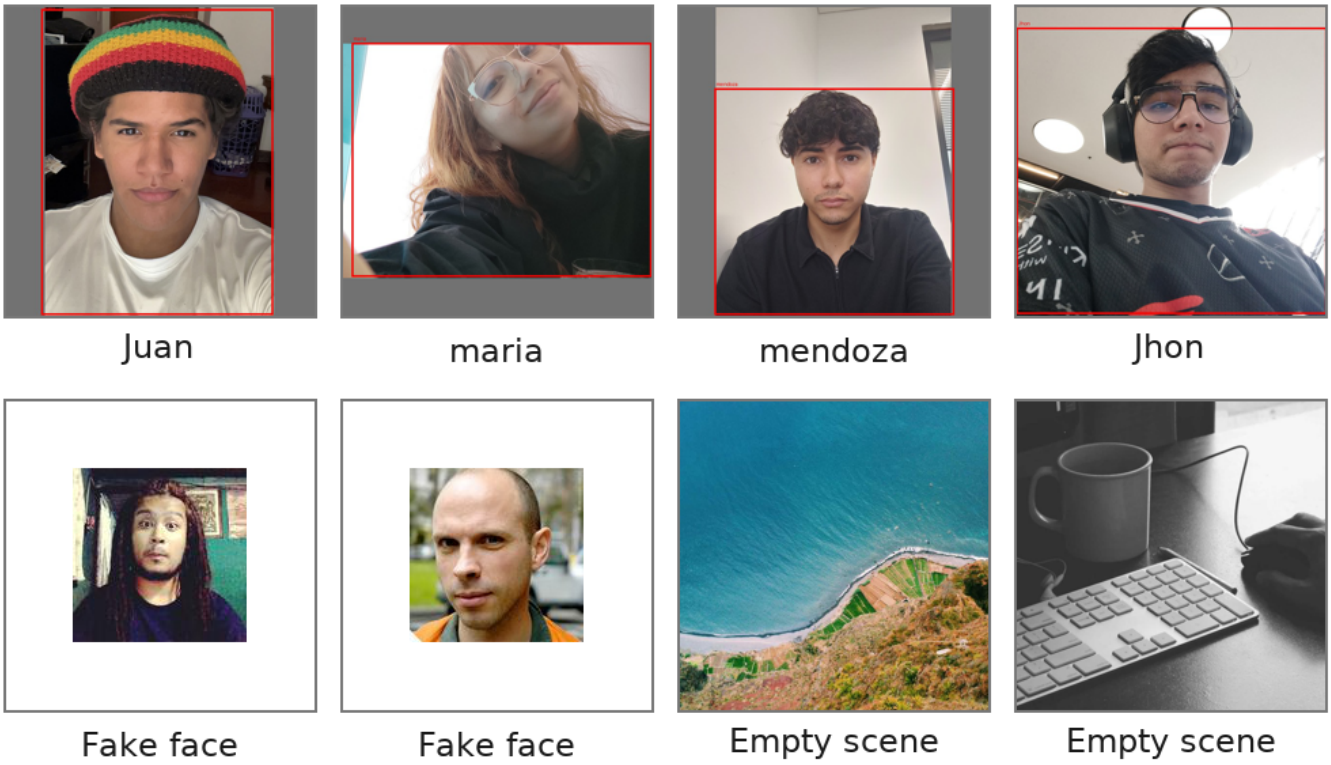

Fig. 1. Representative samples from the base dataset. The top row shows positive identity images with the reviewed YOLO bounding boxes. The bottom row shows the two negative categories used to reduce false positives: fake faces and empty scenes.

`dataset_augmented/train` split, while the 10 validation positives remain unchanged as the validation set. This figure represents the complete offline materialization process used before YOLOv5 training begins.

Fig. 3 shows the same augmentation idea in a smaller example. Starting from a single boxed `Mateo1` image, the pipeline generates multiple boxed outputs that preserve a valid localization target while changing appearance, orientation, and lighting conditions.

Negative samples followed a different rule. Because their label files are intentionally empty, they skip the offline Albumentations stage and are copied unchanged into the augmented train split. This detail is important because the augmented dataset is not a uniform expansion of every image, but a targeted expansion of the scarce labeled positives while preserving the rejection examples that teach the model not to classify every face or every background as a known student. As a result, the final `dataset_augmented` train split contains 1069 images, while the validation split remains at 10 labeled images.

IV. Training

The augmented dataset was used to fine-tune a `yolov5n` model initialized from the public YOLOv5 nano checkpoint. During this stage, YOLOv5 still applied a smaller online augmentation policy, but the aggressive object-detection augmentations that break facial identity, such as Mosaic, MixUp, Copy-Paste, and vertical flip, were disabled. This was important because the model was learning fine-grained identity differences from very few original images, so preserving face geometry was more useful than creating synthetic image compositions.

The final background-and-strangers nano run used the 1069-image augmented training split, the 10-image validation split, AdamW optimization, cosine learning rate scheduling, and the first 10 backbone layers frozen. The classification loss was also weighted more heavily than the default configuration, with `cls = 2.0`, because the main error to reduce was assigning the wrong identity or assigning an enrolled identity to an unknown face. Table I summarizes the training configuration used for the final deployed checkpoint.

V. Evaluation

The final checkpoint was evaluated with YOLOv5 validation on the 10-image validation split. Since this validation set has only one image per identity, the numbers should be interpreted as a compact comparison between project runs rather than as a broad estimate of real-world generalization. Even with that limitation, the metrics are useful because they show whether the model can still localize faces and assign identities after the negative samples and few-shot settings are introduced.

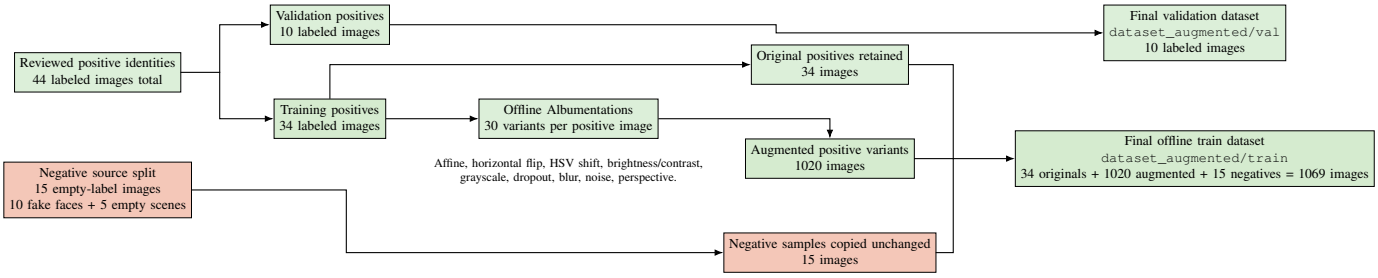

Fig. 2. Full offline augmentation pipeline used to build the final datasets. The initial 44 labeled positive images are split into 34 training positives and 10 validation positives. The 34 training positives are retained and expanded into 1020 augmented positive variants, while the 15 negative samples are copied unchanged into the final 1069-image offline training dataset.

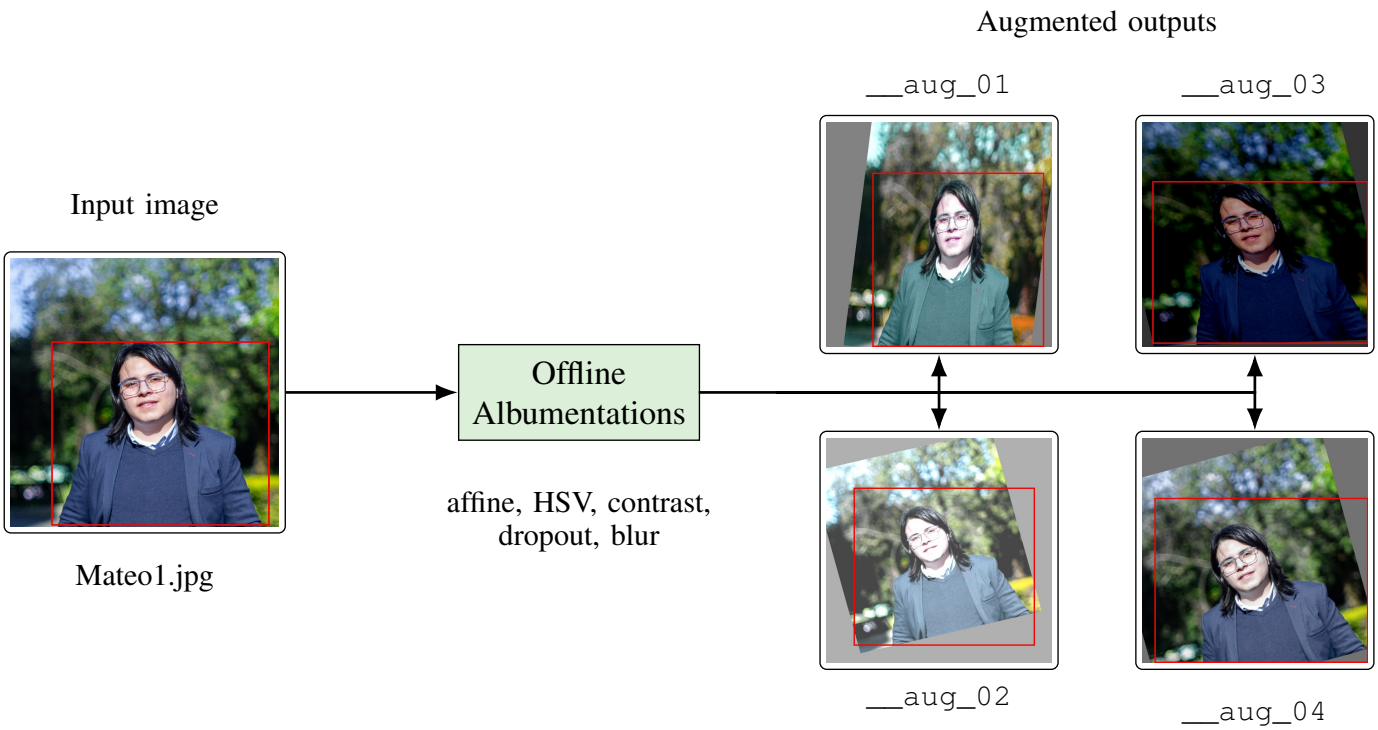

Fig. 3. Example of the offline augmentation pipeline applied to a real `Mateo` training sample. The input image and the four generated outputs are shown with their precomputed YOLO bounding boxes, illustrating that the augmentation process changes the image appearance while preserving valid localization targets.

The selected checkpoint corresponds to the best YOLOv5 fitness score, reached at epoch 99. At that point, the model obtained 0.6098 precision, 0.7688 recall, 0.7960 mAP@0.5, and 0.6965 mAP@0.5:0.95 on the validation split. Fig. 4 reports the main validation values together with the confusion matrix generated by the final run.

TABLE I
TRAINING HYPERPARAMETERS USED FOR THE FINAL YOLOV5N IDENTITY DETECTOR.

Group	Parameter	Value
Model	Initial weights	`yolov5n.pt`
Model	Output artifacts	`best.pt`, `best.onnx`
Dataset	Training split	1069 images from `dataset_augmented/train`
Dataset	Validation split	10 images from `dataset_augmented/val`
Input	Image size	640×640
Training	Epochs and patience	160 requested, patience 50, best checkpoint at epoch 99
Training	Batch size / workers / cache	8 / 4 / RAM cache
Training	Frozen layers	First 10 layers
Optimizer	Optimizer	AdamW
Optimizer	Learning rate schedule	`lr0 = 0.0032`, `lrf = 0.12`, cosine LR enabled
Optimizer	Momentum / weight decay	`momentum = 0.937`, `weight_decay = 0.0005`
Loss	Box / objectness / classification	`box = 0.05`, `obj = 1.0`, `cls = 2.0`
Online augmentation	Enabled	HSV shift, translate = 0.1, scale = 0.2, horizontal flip = 0.5
Online augmentation	Disabled	vertical flip, Mosaic, MixUp, Copy-Paste

Validation metrics

Metric	Value
Best epoch	99
Precision	0.6098
Recall	0.7688
mAP@0.5	0.7960
mAP@0.5:0.95	0.6965
Val box loss	0.004992
Val obj loss	0.004357
Val cls loss	0.070435

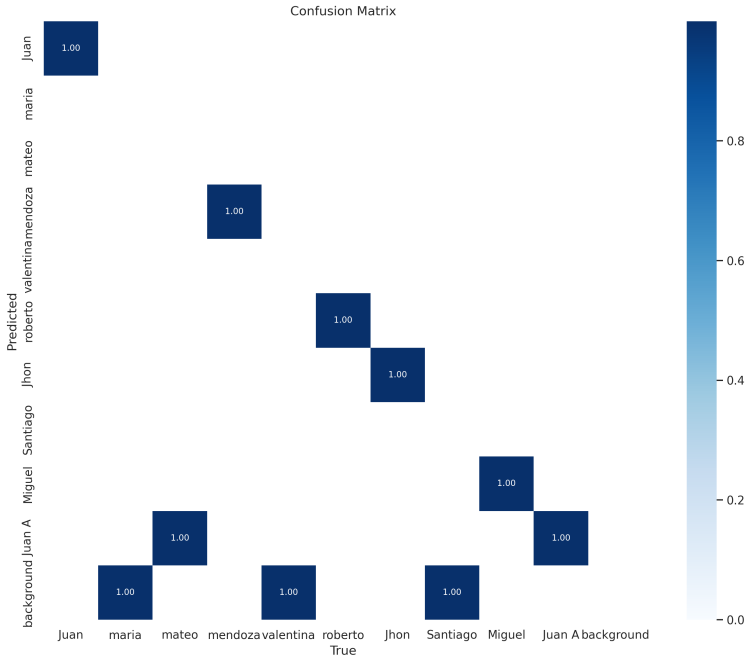

Fig. 4. Validation metrics and confusion matrix generated by YOLOv5 for the final `person_identities_nano_with_bg_and_strangers` run. The matrix summarizes identity predictions on the validation images and includes the background column/row used by YOLOv5 to account for missed or extra detections.

REFERENCES

[1] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf, "DeepFace: Closing the Gap to Human-Level Performance in Face Verification," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2014, pp. 1701–1708.

[2] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 815–823.

[3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 4685–4694.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.

[5] G. Jocher *et al.*, "YOLOv5 by Ultralytics," Zenodo, 2020, doi: 10.5281/zenodo.3908559.

[6] C.-Y. Wang, H.-Y. M. Liao, Y.-H. Wu, P.-Y. Chen, J.-W. Hsieh, and I.-H. Yeh, "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 1571–1580.

[7] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path Aggregation Network for Instance Segmentation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 8759–8768.

[8] A. Buslaev, A. Parinov, E. Khvedchenya, V. I. Iglovikov, and A. A. Kalinin, "Albumentations: Fast and Flexible Image Augmentations," arXiv:1809.06839, 2018, doi: 10.48550/arXiv.1809.06839.